

Act.03 Programación de comunicaciones en red

1. Debes crear una aplicación cliente/servidor que se comunique por un puerto entre el 1800 y 2000 y que realice lo siguiente: El servidor debe generar un número secreto de forma aleatoria entre el 0 y el 200. El objetivo del cliente es solicitarle al usuario un número y enviarlo al servidor hasta que adivine el número secreto. Para ello, el servidor para cada número que le envía el cliente le indicará si es menor, mayor o es el número secreto del servidor.

Captura código clase Cliente:

```
public class Cliente { @ TomasPerezC21 *

    public static void main(String[] args) { @ TomasPerezC21 *

        String direccionServidor = "localhost";
        int puertoServidor = 1900;

        Socket socketCliente = null;
        DataInputStream servidorEntrada = null;
        DataOutputStream servidorSalida = null;
        Scanner sc = new Scanner(System.in);

        try {
            socketCliente = new Socket(direccionServidor,puertoServidor);

            servidorEntrada = new DataInputStream(socketCliente.getInputStream());
            servidorSalida = new DataOutputStream(socketCliente.getOutputStream());

            int numUsuario;

            boolean acierto = false;

            while(!acierto){
                System.out.println("Cliente, introduce un número: ");
                numUsuario = solicitarNumUsuario(sc);
                servidorSalida.writeInt(numUsuario);
                int respuestaServer = servidorEntrada.readInt();
                if(respuestaServer == 1){
                    System.out.println("El número " + numUsuario + " es mayor al número secreto.");
                }
                else if(respuestaServer == 2){
                    System.out.println("El número " + numUsuario + " es menor al número secreto.");
                }
                else if(respuestaServer == 3){
                    System.out.println("Enhorabuena, has acertado el número secreto.");
                    acierto = true;
                }
            }
            servidorEntrada.close();
            servidorSalida.close();
            socketCliente.close();
            sc.close();

        } catch (IOException e) {
            throw new RuntimeException(e);
        }
    }

    private static int solicitarNumUsuario(Scanner sc) { return sc.nextInt(); }
}
```

Captura clase Servidor:

```
public class Servidor {  TomasPerezC21 *

    public static void main(String[] args) {  TomasPerezC21 *
        Random rand = new Random();
        int puertoServidor = 1900;

        System.out.println("GENERANDO NÚMERO SECRETO EN SERVIDOR...");
        int secreto = generarNumeroSecreto(rand);

        ServerSocket socketServidor = null;
        Socket socketCliente = null;
        DataInputStream clienteEntrada = null;
        DataOutputStream clienteSalida = null;

        try{
            socketServidor = new ServerSocket(puertoServidor);
            socketCliente = socketServidor.accept();
            clienteEntrada = new DataInputStream(socketCliente.getInputStream());
            clienteSalida = new DataOutputStream(socketCliente.getOutputStream());

            int numeroCliente;

            boolean acierto = false;

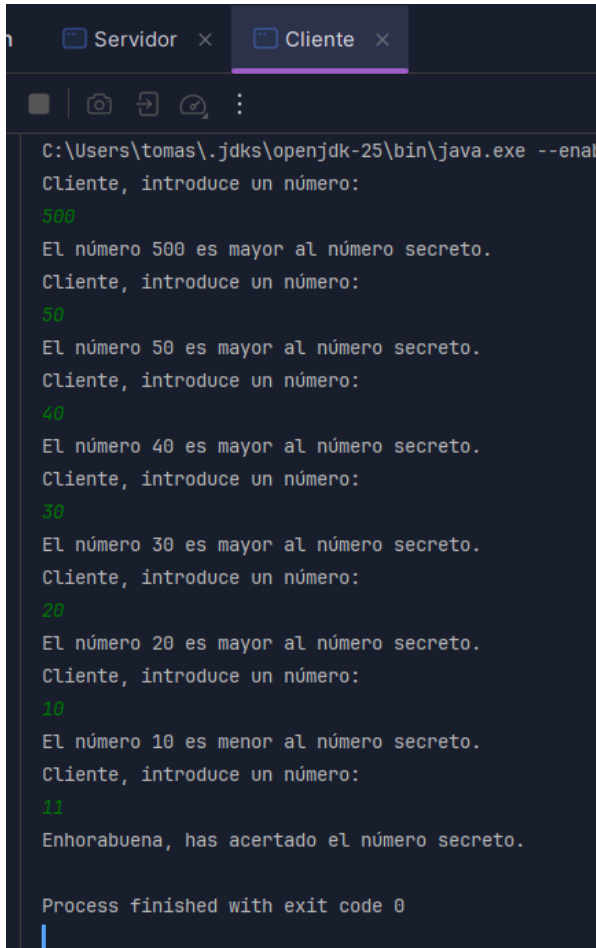
            while(!acierto){
                numeroCliente = clienteEntrada.readInt();
                if (numeroCliente > secreto){
                    clienteSalida.writeInt( v: 1);
                }
                else if(numeroCliente < secreto){
                    clienteSalida.writeInt( v: 2);
                }
                else {
                    clienteSalida.writeInt( v: 3);
                    acierto = true;
                }
            }

        }catch(IOException e){
            System.err.println("Error en servidor");
        }

        System.out.println("Juego terminado. Número secreto: " + secreto);
    }

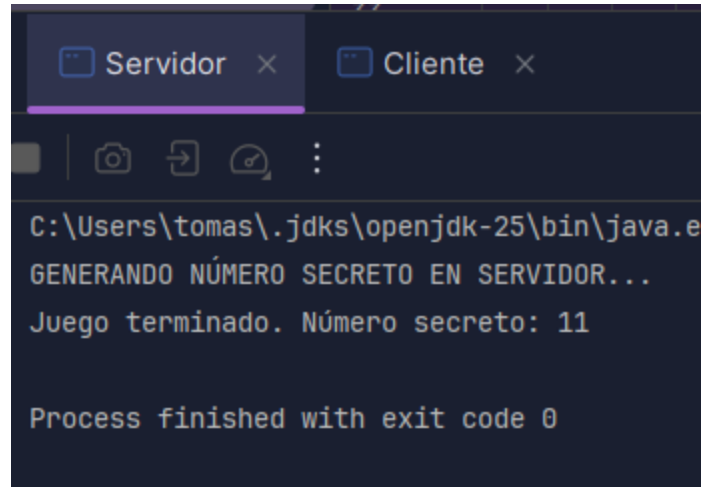
    private static int generarNumeroSecreto(Random rand) { 1 usage  TomasPerezC21
        int numeroSecreto = rand.nextInt( origin: 0, bound: 201);
        return numeroSecreto;
    }
}
```

Salida por terminal:



```
C:\Users\tomas\.jdk\openjdk-25\bin\java.exe --enab
Cliente, introduce un número:
500
El número 500 es mayor al número secreto.
Cliente, introduce un número:
50
El número 50 es mayor al número secreto.
Cliente, introduce un número:
40
El número 40 es mayor al número secreto.
Cliente, introduce un número:
30
El número 30 es mayor al número secreto.
Cliente, introduce un número:
20
El número 20 es mayor al número secreto.
Cliente, introduce un número:
10
El número 10 es menor al número secreto.
Cliente, introduce un número:
11
Enhorabuena, has acertado el número secreto.

Process finished with exit code 0
```



```
C:\Users\tomas\.jdk\openjdk-25\bin\java.e
GENERANDO NÚMERO SECRETO EN SERVIDOR...
Juego terminado. Número secreto: 11

Process finished with exit code 0
```

Comentarios adicionales:

Empezando por la clase Servidor, se ha generado el número aleatorio entre 0 y 200 en un método estático fuera del main. La lógica “potente” reside en el bucle *while*, condicionado por un booleano inicializado en *false* para asegurar que entre ha dicho bucle y así realizar las comprobaciones del número introducido en la clase Cliente. Si el número del cliente es mayor al generado aleatoriamente (secreto) el servidor devuelve el valor “1” al cliente, si es menor devuelve “2” y “3” cuando el número sea el esperado. El bucle termina cuando la última condición se cumpla, y mostramos en la consola del servidor el número secreto para así comprobar luego en la consola del Cliente que es el correcto. Por otro lado, la clase Cliente va enviando números a la clase Servidor. Esta mostrará un mensaje en base a los números devueltos por el servidor (1, 2, 3) como se ha mencionado anteriormente.

2. Modifica la actividad para que el servidor permita trabajar de forma concurrente con varios clientes.

Para este ejercicio pueden haber dos formas de afrontarlo. Una sería que hay un número secreto diferente para cada cliente, y la otra sería que todos los clientes comparten el mismo número secreto. Yo he optado por la segunda, para ello he generado el número secreto en el Servidor y enviado al constructor de la clase GestorClientes.

Captura clase ServidorMulti:

```
public class ServidorMulti { new *  
|  
    public static void main(String[] args) { new *  
  
        Random aleatorio = new Random();  
  
        int puertoServidor = 2900;  
        ServerSocket socket = null;  
        Socket socketCliente = null;  
  
        int secreto = aleatorio.nextInt( origin: 0, bound: 201);  
  
        try {  
            socket = new ServerSocket(puertoServidor);  
  
            while(true){  
                socketCliente = socket.accept();  
                GestorClientes gp = new GestorClientes(socketCliente, secreto);  
                new Thread(gp).start();  
            }  
  
        } catch (IOException e) {  
            throw new RuntimeException(e);  
        }  
    }  
}
```

Captura clase Cliente:

```
public class Cliente { new *  
  
    public static void main(String[] args) { new *  
  
        String direccionServidor = "localhost";  
        int puertoServidor = 2900;  
  
        Socket socketCliente = null;  
        DataInputStream servidorEntrada = null;  
        DataOutputStream servidorSalida = null;  
        Scanner sc = new Scanner(System.in);  
  
        try {  
            socketCliente = new Socket(direccionServidor,puertoServidor);  
  
            servidorEntrada = new DataInputStream(socketCliente.getInputStream());  
            servidorSalida = new DataOutputStream(socketCliente.getOutputStream());  
  
            int numUsuario;  
  
            boolean acierto = false;  
  
            String inicioJuego = servidorEntrada.readUTF();  
            System.out.println(inicioJuego);  
  
            while(!acierto){  
                System.out.println("Cliente, introduce un número: ");  
                numUsuario = solicitarNumUsuario(sc);  
                servidorSalida.writeInt(numUsuario);  
  
                int respuestaServer = servidorEntrada.readInt();  
  
                if(respuestaServer == 1){  
                    System.out.println("Te has pasado (El secreto es menor a " + numUsuario + ")");  
                }  
                else if(respuestaServer == 2){  
                    System.out.println("Te has quedado corto (El secreto es mayor a " + numUsuario + ")");  
                }  
                else if(respuestaServer == 3){  
                    System.out.println("Enhorabuena, has acertado el número secreto.");  
                    acierto = true;  
                }  
            }  
            servidorEntrada.close();  
            servidorSalida.close();  
            socketCliente.close();  
            sc.close();  
  
        } catch (IOException e) {  
            throw new RuntimeException(e);  
        }  
    }  
}
```

Captura clase GestorClientes:

```
public class GestorClientes implements Runnable{ 2 usages new *
    private Socket socketCliente; 4 usages
    private int idCliente; 4 usages
    private static int NUMCLIENTE = 0; 2 usages

    private int secretoServidor; 3 usages

    private final static Object candado = new Object(); 1 usage

    public GestorClientes(Socket socketCliente, int secretoServidor) { 1 usage new *
        this.socketCliente = socketCliente;
        synchronized (candado){
            NUMCLIENTE++;
            this.idCliente = NUMCLIENTE;
        }
        this.secretoServidor = secretoServidor;
    }

    @Override new *
    public void run() {

        DataInputStream clienteEntrada = null;
        DataOutputStream clienteSalida = null;

        try {
            clienteEntrada = new DataInputStream(socketCliente.getInputStream());
            clienteSalida = new DataOutputStream(socketCliente.getOutputStream());

            boolean acierto = false;

            int numCliente;

            clienteSalida.writeUTF("El cliente con id " + idCliente + " empieza a jugar.");

            while (!acierto){
                numCliente = clienteEntrada.readInt();

                System.out.println("El cliente " + idCliente + " dice: " + numCliente);
                if (numCliente > secretoServidor) {
                    clienteSalida.writeInt(1);
                } else if (numCliente < secretoServidor) {
                    clienteSalida.writeInt(2);
                } else {
                    clienteSalida.writeInt(3);
                    acierto = true;
                }
            }

            System.out.println("El juego ha terminado para el jugador: "+idCliente+" .");
            clienteEntrada.close();
            clienteSalida.close();
            socketCliente.close();

        } catch (IOException e) {
            throw new RuntimeException(e);
        }
    }
}
```

Salida por terminal:

```
ServidorMulti x ClienteMulti x ClienteMulti x ClienteMulti x
C:\Users\tomas\.jdk\openjdk-25\bin\java.exe --enable-preview "-javaagent:C
El cliente 1 dice: 68
El cliente 2 dice: 12
El cliente 3 dice: 45
El cliente 1 dice: 21
El cliente 1 dice: 45
El cliente 1 dice: 40
El cliente 1 dice: 35
El cliente 1 dice: 36
El cliente 1 dice: 37
El cliente 1 dice: 38
El juego ha terminado para el jugador: 1 .
El cliente 2 dice: 56
El cliente 2 dice: 45
El cliente 3 dice: 77
El cliente 3 dice: 21
El cliente 3 dice: 33
```

```
C:\Users\tomas\.jdk\openjdk-25\bin\java.exe --enab
El cliente con id 1 empieza a jugar.
Cliente, introduce un número:
68
Te has pasado (El secreto es menor a 68)
Cliente, introduce un número:
21
Te has quedado corto (El secreto es mayor a 21)
Cliente, introduce un número:
45
Te has pasado (El secreto es menor a 45)
Cliente, introduce un número:
40
Te has pasado (El secreto es menor a 40)
Cliente, introduce un número:
35
Te has quedado corto (El secreto es mayor a 35)
Cliente, introduce un número:
36
Te has quedado corto (El secreto es mayor a 36)
Cliente, introduce un número:
37
Te has quedado corto (El secreto es mayor a 37)
Cliente, introduce un número:
38
Enhorabuena, has acertado el número secreto.

Process finished with exit code 0
```

Comentarios adicionales:

He simplificado la parte de la salida por terminal ya que es la misma para todos los clientes. El ejercicio es muy similar al primero salvo que para gestionar los clientes se crea una clase “worker” que se encarga de gestionar la comunicación exclusiva con ese cliente específico en un hilo de ejecución paralelo, liberando al Servidor para que pueda seguir aceptando a más gente. Para el desarrollo de la clase gestora me he guiado de los ejemplos que hemos visto en clase.

3. Mejorar la aplicación vista en el apartado 2 de los apuntes (Traducción de palabras). Novedades a incluir: debe ser multihilo (permitir trabajar de forma concurrente con varios clientes), el número de palabras con el que trabaja el servidor sean leídas de un fichero (este fichero debe tener al menos 10 líneas de 10 palabras). Posterior a la palabra y en la misma línea debe tener las diferentes traducciones válidas de dicha palabra. Por ejemplo, si la palabra es Adiós la traducción válida podría ser tanto Bye como GoodBye, por lo que la línea del fichero debería tener el siguiente contenido “Adiós Bye GoodBye”. El servidor le tiene que enviar todas las palabras del fichero (intentar que no siempre se envíen en el mismo orden) y obtendrá las respuestas del cliente. Una vez que todas se han enviado le enviará un resumen de palabras acertadas respecto del total.

La parte de la salida por terminal la he intentado reducir lo máximo para que no ocupe demasiado espacio. Aún así el flujo del programa considero que se entiende bien. Además, las capturas de pantalla de la clase GestorPalabras las he hecho por partes con el fin de facilitarte la revisión, ya que es bastante más densa que las demás.

Captura clase ServidorPalabrasMulti:

```
public class ServidorPalabrasMulti { new *

    private static String RUTAFICHERO = "src/Ejercicio3/PalabrasTraducir"; 1 usage

    public static void main(String[] args) { new *

        int puertoServidor = 2500;
        ServerSocket socket = null;
        Socket socketCliente = null;

        ArrayList<String> datosFichero = obtenerDatosFichero();

        try {
            socket = new ServerSocket(puertoServidor);

            while (true) {
                //Cada cliente recibe las palabras del fichero en diferente orden
                Collections.shuffle(datosFichero);

                socketCliente = socket.accept();
                GestorPalabras gp = new GestorPalabras(socketCliente, datosFichero);
                new Thread(gp).start();
            }
        } catch (IOException e) {
            throw new RuntimeException(e);
        }
    }

    private static ArrayList<String> obtenerDatosFichero(){ 1 usage new *
        ArrayList<String> palabras = new ArrayList<>();

        try {
            BufferedReader br = new BufferedReader(new FileReader(RUTAFICHERO));

            String linea;

            while ((linea = br.readLine()) != null) {
                palabras.add(linea);
            }

            Collections.shuffle(palabras);
        } catch (Exception e) {
            throw new RuntimeException(e);
        }

        return palabras;
    }
}
```

Como no se especifica en la tarea el fichero lo he rellenado manualmente y añadido a la ruta.

Clase Cliente:

```
public class Cliente { new *

    public static void main(String[] args) { new *

        String direccionServidor = "localhost";
        int puertoServidor = 2500;

        Socket socketCliente = null;
        DataInputStream entradaDelServidor = null;
        DataOutputStream salidaAlServidor = null;
        Scanner sc = new Scanner(System.in);

        try {
            socketCliente = new Socket(direccionServidor,puertoServidor);

            entradaDelServidor = new DataInputStream(socketCliente.getInputStream());
            salidaAlServidor = new DataOutputStream(socketCliente.getOutputStream());

            String palabra = entradaDelServidor.readUTF();
            while(!palabra.equalsIgnoreCase( anotherString: "fin")){

                System.out.println("Debes traducir la palabra: " + palabra);
                String respuesta = sc.nextLine();
                salidaAlServidor.writeUTF(respuesta);
                palabra = entradaDelServidor.readUTF();
            }

            System.out.println("Ya no hay que traducir más palabras.");
            entradaDelServidor.close();
            salidaAlServidor.close();
            socketCliente.close();
            sc.close();

        } catch (IOException e) {
            throw new RuntimeException(e);
        }
    }
}
```

Captura clase GestorPalabras:

Atributos y constructor:

```
public class GestorPalabras implements Runnable{ 2 usages new *  
  
    private Socket socketCliente; 4 usages  
    private int idCliente; 4 usages  
    private ArrayList<String> copiaPalabrasFichero; 4 usages  
  
    private final Object candado = new Object(); 1 usage  
    private static int NUMCLIENTE = 0; 2 usages  
  
    private static int NUMEROPALABRAS = 0; 2 usages  
  
    public GestorPalabras(Socket socketCliente, ArrayList<String> palabrasFicheroServidor) {  
        this.socketCliente = socketCliente;  
        this.copiaPalabrasFichero = new ArrayList<>(palabrasFicheroServidor);  
        NUMEROPALABRAS = palabrasFicheroServidor.size();  
  
        synchronized (candado) {  
            NUMCLIENTE++;  
            this.idCliente = NUMCLIENTE;  
        }  
    }  
}
```

Me he basado en el ejercicio que hicimos en clase. El ejercicio lo he realizado sobre la marcha utilizando lo que se me iba ocurriendo, seguramente haya alguna forma de hacerlo todo más simple.

Método Run con la lógica:

```
@Override new *
public void run() {

    DataInputStream entradaDelCliente = null;
    DataOutputStream salidaAlCliente = null;

    ArrayList<String> palabrasEsp = obtenerPalabrasEsp(copiaPalabrasFichero);
    ArrayList<String> palabrasIngles = obtenerPalabrasIngles(copiaPalabrasFichero);

    int contadorAciertos = 0;
    ArrayList<String> resumenFinal = new ArrayList<>();

    try {
        entradaDelCliente = new DataInputStream(socketCliente.getInputStream());
        salidaAlCliente = new DataOutputStream(socketCliente.getOutputStream());

        while(!palabrasEsp.isEmpty()){

            String palabraActualEsp = palabrasEsp.removeFirst();
            String solucionesIngles = palabrasIngles.removeFirst();
            String lineaActual = copiaPalabrasFichero.removeFirst();

            salidaAlCliente.writeUTF(palabraActualEsp);
            System.out.println("Le he enviado al jugador: " + idCliente + " la palabra: "+palabraActualEsp);

            String respuestaCliente = entradaDelCliente.readUTF();
            System.out.println("El jugador " + idCliente + " responde: "+respuestaCliente);

            //Validación de respuesta de usuario para resumen final del juego
            if(comprobarRespuestaCliente(solucionesIngles,respuestaCliente)){
                contadorAciertos++;
                resumenFinal.add(lineaActual);
            }
        }

        //Para avisar al cliente de que el juego se ha acabado
        String fin = "fin";
        salidaAlCliente.writeUTF(fin);

        System.out.println("RESUMEN FINAL " + " Jugador " + idCliente + ". Total de aciertos: " + contadorAciertos + " / " + NUMEROPALABRAS);
        for (String s : resumenFinal) {
            System.out.println("Acertada: " + s);
        }

        entradaDelCliente.close();
        salidaAlCliente.close();
        socketCliente.close();
    } catch (IOException e) {
        throw new RuntimeException(e);
    }
}
```

He separado ciertas funcionalidades por métodos auxiliares que se verán en la siguiente captura de pantalla. Básicamente la forma en la que he planteado el ejercicio ha sido en separar las palabras en español de las palabras en inglés. A continuación, se comprobará la respuesta del cliente en el método “comprobarRespuestaCliente” de cara al resumen final una vez se acaben las palabras disponibles en español para traducir. Además, para avisarle al cliente de que el juego se acaba le mando la palabra “fin”.

Métodos privados auxiliares:

```
private ArrayList<String> obtenerPalabrasEsp(ArrayList<String> palabrasFichero) { 1 usage new *  
  
    ArrayList<String> palabrasEsp = new ArrayList<>();  
  
    for (String linea : palabrasFichero) {  
        String[] palabras = linea.split( regex: " ");  
        palabrasEsp.add(palabras[0]);  
    }  
    return palabrasEsp;  
}  
  
private ArrayList<String> obtenerPalabrasIngles(ArrayList<String> palabrasFichero) { 1 usage new *  
  
    ArrayList<String> palabrasIngles = new ArrayList<>();  
  
    for (String linea : palabrasFichero) {  
        String[] palabras = linea.split( regex: " ");  
        String palabrasInglesConcatenadas = palabras[1] + " " + palabras[2];  
        palabrasIngles.add(palabrasInglesConcatenadas);  
    }  
    return palabrasIngles;  
}  
  
private boolean comprobarRespuestaCliente(String palabrasIngles, String respuestaCliente) { 1 usage new *  
    String[] palabras = palabrasIngles.split( regex: " ");  
    return respuestaCliente.equalsIgnoreCase(palabras[0]) || respuestaCliente.equalsIgnoreCase(palabras[1]);  
}
```

Lo he hecho de esta forma para intentar mejorar la legibilidad general del código e intentar dejarlo lo mejor posible pero eso, seguramente haya otras formas de resolver el ejercicio.

Salida por terminal:

```
ServidorPalabrasMulti x Cliente3 x Cliente3
C:\Users\tomas\.jdk\openjdk-25\bin\java.exe --enabl
Le he enviado al jugador: 1 la palabra: Amor
Le he enviado al jugador: 2 la palabra: Tiempo
El jugador 1 responde: LOVE
Le he enviado al jugador: 1 la palabra: Dinero
El jugador 1 responde: MONEY
Le he enviado al jugador: 1 la palabra: Paz
El jugador 1 responde: PEACE
Le he enviado al jugador: 1 la palabra: Tiempo
El jugador 1 responde: A
Le he enviado al jugador: 1 la palabra: Mago
El jugador 1 responde: A
Le he enviado al jugador: 1 la palabra: Adiós
El jugador 1 responde: A
Le he enviado al jugador: 1 la palabra: Fuego
El jugador 1 responde: FIRE
Le he enviado al jugador: 1 la palabra: Camino
El jugador 1 responde: ROAD
Le he enviado al jugador: 1 la palabra: Sueño
El jugador 1 responde: A
Le he enviado al jugador: 1 la palabra: Guapo
El jugador 1 responde: HANDSOME
RESUMEN FINAL Jugador 1. Total de aciertos: 6 / 10
Acertada: Amor Love Heart
Acertada: Dinero Money Cash
Acertada: Paz Peace Calm
Acertada: Fuego Fire Flame
Acertada: Camino Path Road
Acertada: Guapo Beautiful Handsome
```

```
El jugador 2 responde: time
Le he enviado al jugador: 2 la palabra: Adiós
El jugador 2 responde: bye
Le he enviado al jugador: 2 la palabra: Guapo
El jugador 2 responde: a
Le he enviado al jugador: 2 la palabra: Dinero
El jugador 2 responde: a
Le he enviado al jugador: 2 la palabra: Fuego
El jugador 2 responde: a
Le he enviado al jugador: 2 la palabra: Sueño
El jugador 2 responde: a
Le he enviado al jugador: 2 la palabra: Mago
El jugador 2 responde: a
Le he enviado al jugador: 2 la palabra: Amor
El jugador 2 responde: love
Le he enviado al jugador: 2 la palabra: Camino
El jugador 2 responde: road
Le he enviado al jugador: 2 la palabra: Paz
El jugador 2 responde: peace
RESUMEN FINAL Jugador 2. Total de aciertos: 5 / 10
Acertada: Tiempo Time Clock
Acertada: Adiós Bye Goodbye
Acertada: Amor Love Heart
Acertada: Camino Path Road
Acertada: Paz Peace Calm
```

```
ServidorPalabrasMulti x Cliente3 x C
C:\Users\tomas\.jdk\openjdk-25\bin\java.exe
Debes traducir la palabra: Amor
LOVE
Debes traducir la palabra: Dinero
MONEY
Debes traducir la palabra: Paz
PEACE
Debes traducir la palabra: Tiempo
A
Debes traducir la palabra: Mago
A
Debes traducir la palabra: Adiós
A
Debes traducir la palabra: Fuego
FIRE
Debes traducir la palabra: Camino
ROAD
Debes traducir la palabra: Sueño
A
Debes traducir la palabra: Guapo
HANDSOME
Ya no hay que traducir más palabras.

Process finished with exit code 0
```

```
ServidorPalabrasMulti x Cliente3 x Clien
C:\Users\tomas\.jdk\openjdk-25\bin\java.exe --e
Debes traducir la palabra: Tiempo
time
Debes traducir la palabra: Adiós
bye
Debes traducir la palabra: Guapo
a
Debes traducir la palabra: Dinero
a
Debes traducir la palabra: Fuego
a
Debes traducir la palabra: Sueño
a
Debes traducir la palabra: Mago
a
Debes traducir la palabra: Amor
love
Debes traducir la palabra: Camino
road
Debes traducir la palabra: Paz
peace
Ya no hay que traducir más palabras.

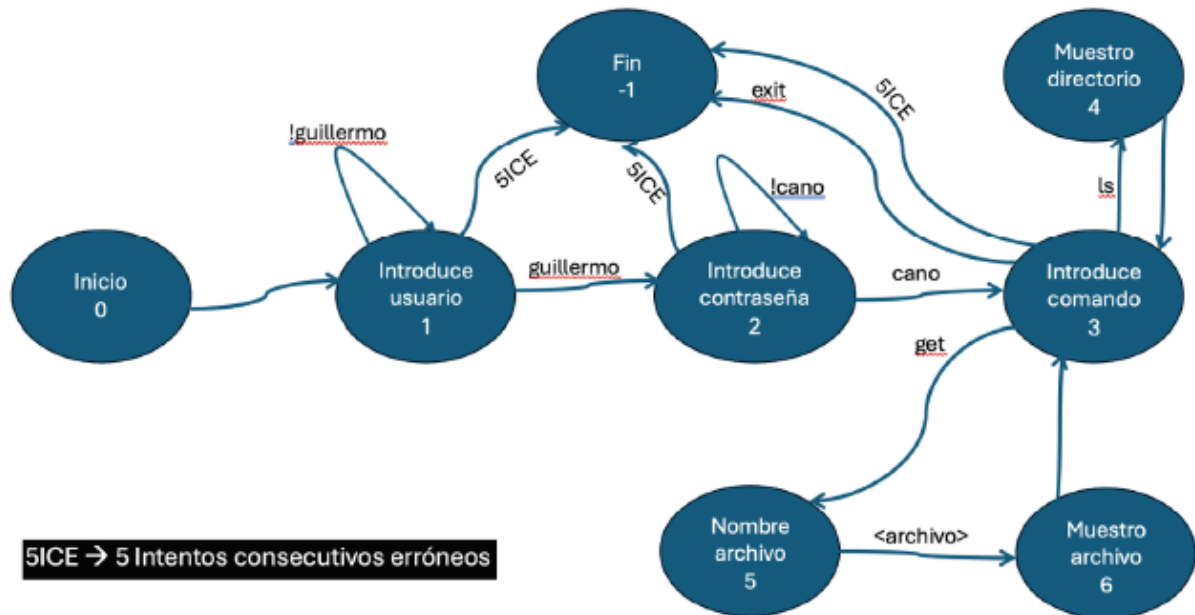
Process finished with exit code 0
```

Fichero de texto:

Adiós	Bye	Goodbye
Dinero	Money	Cash
Sueño	Dream	Wish
Camino	Path	Road
Fuego	Fire	Flame
Mago	Magician	Wizard
Tiempo	Time	Clock
Guapo	Beautiful	Handsome
Paz	Peace	Calm
Amor	Love	Heart

El ejercicio me ha gustado bastante. Ha sido entretenido.

4. Crea un servidor que una vez iniciada sesión a través de un nombre de usuario y contraseña específico (en el ejemplo del diagrama de estados sería guillermo / cano) el sistema permita ver el contenido del directorio actual, mostrar el contenido de un determinado archivo y salir. A continuación, puedes ver un diagrama de estados que muestra el funcionamiento del servidor.



Capturas de pantalla del código clase Servidor:

```
public class Servidor {  @ Tomás *

    private static final String USUARIO = "Tomas";  1 usage
    private static final String CLAVE = "1234";  1 usage
    private static int numIntentos = 5;  5 usages
    private static String nombreArchivoSolicitado = "";  2 usages

    public static void main(String[] args) {  @ Tomás *

        int puertoServidor = 2100;

        int estadoServidor = 0;

        ServerSocket socketServidor = null;
        Socket socketCliente = null;
        DataInputStream reciboCliente = null;
        DataOutputStream envioCliente = null;

        try{
            socketServidor = new ServerSocket(puertoServidor);
            socketCliente = socketServidor.accept();
            reciboCliente = new DataInputStream(socketCliente.getInputStream());
            envioCliente = new DataOutputStream(socketCliente.getOutputStream());

            System.out.println("Servidor iniciado. Esperando credenciales de inicio de sesión...");
```

Lógica del servidor en while con switch para los estados del servidor:

```
while(estadoServidor != -1){

    switch (estadoServidor){
        case 0:
            estadoServidor = 1;
            break;

        case 1:
            String usuarioCliente = reciboCliente.readUTF(); //leemos usuario

            if(comprobarUsuario(usuarioCliente)){
                numIntentos = 5; //Si el usuario es correcto se reinician los intentos a 5
                estadoServidor = 2; //Actualizamos el estado para avanzar a la contraseña
                envioCliente.writeInt(estadoServidor); //Pasamos a solicitar contraseña
            }else {
                numIntentos--; //Si el usuario no es correcto se van restando intentos.
                envioCliente.writeInt(estadoServidor); //Seguimos en estado1 pidiendo usuario hasta acabar intentos
            }
            break;

        case 2:
            String claveCliente = reciboCliente.readUTF(); //leemos clave
            if(comprobarClave(claveCliente)){
                numIntentos = 5;
                estadoServidor = 3;
                envioCliente.writeInt(estadoServidor); //El estado avanza y pasamos a solicitar comando
            }else {
                numIntentos--;
                envioCliente.writeInt(estadoServidor); //Notificamos que siga introduciendo contraseña
            }
            break;
```

```
case 3:

    System.out.println("Esperando comando del cliente...");
    String comandoCliente = reciboCliente.readUTF().toLowerCase();

    switch(comandoCliente) {
        case "ls":
            // El diagrama dice que si es "ls", vamos al estado 4
            estadoServidor = 4;
            break;

        case "get":
            // El diagrama dice que si es "get", vamos al estado 5
            estadoServidor = 5;
            break;

        case "exit":
            // El diagrama dice que si es "exit", vamos a Fin (-1)
            estadoServidor = -1;
            break;

        default:
            //Si escribe algo raro, le avisamos y nos quedamos en el estado 3
            envioCliente.writeUTF(str: "Comando desconocido. Usa: ls, get, exit");
            break;
    }
    break;

case 4:

    System.out.println("Listando directorio...");
    File carpeta = new File(pathname: "."); // directorio actual
    String[] listaArchivos = carpeta.list();

    StringBuilder mensajeAEnviar = new StringBuilder();
    if (listaArchivos != null) {
        for (String fichero : listaArchivos) {
            mensajeAEnviar.append(fichero).append("\n"); // Añadimos salto de línea
        }
    }

    envioCliente.writeUTF(mensajeAEnviar.toString()); // Enviamos el listado al cliente

    estadoServidor = 3;
    break;
```

```
        case 5:
            //Recibimos el nombre y pasamos al estado 6 para mostrar contenido
            nombreArchivoSolicitado = reciboCliente.readUTF();
            estadoServidor = 6;

            break;

        case 6:
            File archivo = new File(nombreArchivoSolicitado);

            if(archivo.exists()){
                BufferedReader br = new BufferedReader(new FileReader(archivo));
                StringBuilder mensaje = new StringBuilder();
                String linea;
                while ((linea = br.readLine()) != null) {
                    mensaje.append(linea).append("\n");
                }

                envioCliente.writeUTF(mensaje.toString()); //Envío de contenido del archivo al cliente
                br.close();
            }else{
                envioCliente.writeUTF("Error. Archivo no encontrado.");
            }

            estadoServidor = 3; //volvemos al 3 (principio de los comandos pero con sesión iniciada)
            break;
    }

    //Si los intentos se acaban se cierra la sesión forzando el estadoServidor a -1
    if(numIntentos == 0){
        System.out.println("Sesión errónea. Intentos agotados.");
        envioCliente.writeInt(-1);
        estadoServidor = -1;
    }
}

} catch (IOException e) {
    throw new RuntimeException(e);
}
```

Métodos auxiliares simples:

```
private static boolean comprobarUsuario(String usuario) { return usuario.equals(USUARIO); }

private static boolean comprobarClave(String clave) { return clave.equals(CLAVE); }
```

La clase servidor es la que más chicha tiene ya que he tratado de seguir el diagrama de la foto del ejercicio para el tema de controlar los estados del servidor. Al principio lo estaba haciendo con variables de texto simples pero es mejor tratarlo con enteros ya que se simplifica bastante la comunicación con el cliente. He intentado dejar el ejercicio bastante comentado ya que es complicado explicar todo aquí en la memoria. Básicamente, en base al estado el programa va haciendo una cosa u otra usando las respuestas del cliente.

Capturas de pantalla clase cliente:

```
public class Cliente {  🧑 Tomás *  
  
    public static void main(String[] args) {  new *  
  
        String host = "localhost";  
        int puerto = 2100;  
  
        Socket socketCliente = null;  
        DataInputStream reciboServidor = null;  
        DataOutputStream envioServidor = null;  
        Scanner sc = new Scanner(System.in);  
  
        try {  
            socketCliente = new Socket(host,puerto);  
  
            reciboServidor = new DataInputStream(socketCliente.getInputStream());  
            envioServidor = new DataOutputStream(socketCliente.getOutputStream());  
  
            // Variable local para controlar en qué paso de la autenticación estamos  
            // Inicialmente asumimos que el servidor está en estado 1 (pidiendo usuario)  
            int estadoActual = 1;
```

Lógica de la comunicación servidor-cliente en bucle while:

```
// Login del usuario en base a las credenciales especificadas en el servidor.
while (estadoActual != 3) {

    if (estadoActual == 1) {
        // El servidor espera un USUARIO
        System.out.print("Introduce usuario: ");
        String usuario = sc.nextLine();
        envioServidor.writeUTF(usuario);

        // Leemos qué dice el servidor (nos devolverá 1 si fallamos, 2 si acertamos)
        estadoActual = reciboServidor.readInt();

        if (estadoActual == 1) {
            System.out.println("Usuario incorrecto. Vuelve a intentarlo");
        }
    } else if (estadoActual == 2) {
        // El servidor espera una CONTRASEÑA
        System.out.print("Introduce contraseña: ");
        String clave = sc.nextLine();
        envioServidor.writeUTF(clave);

        // Leemos respuesta (2 si fallamos, 3 si acertamos)
        estadoActual = reciboServidor.readInt();

        if (estadoActual == 2) {
            System.out.println("Contraseña incorrecta. Vuelve a intentarlo");
        }
    }

    // Si el servidor nos devuelve -1 es que se acabaron los intentos
    if (estadoActual == -1) {
        System.out.println("Has agotado los intentos. Conexión cerrada.");
        return;
    }
}
```

Este while comprueba que el usuario y la contraseña estén bien. Si el cliente se equivoca 5 veces el programa se cierra (lógica del servidor). Una vez el registro sea válido el programa entra en la parte de los archivos con los comandos:

```
System.out.println(";Login correcto! Bienvenido.");

//Inicio de sesion correcto ahora la parte de los comandos
boolean salir = false;
while (!salir) {
    System.out.println("\nEscribe un comando (ls, get, exit):");
    System.out.print("> ");
    String comando = sc.nextLine();

    // Enviamos el comando al servidor
    envioServidor.writeUTF(comando);

    switch (comando.toLowerCase()) {
        case "ls":
            // El servidor nos enviará la lista de archivos del directorio en el que nos encontramos
            String respuestaLs = reciboServidor.readUTF();
            System.out.println("--- Archivos del Servidor ---");
            System.out.println(respuestaLs);
            break;

        case "get":
            // El protocolo dice que ahora debemos enviar el nombre del archivo
            System.out.print("Introduce el nombre del archivo: ");
            String archivo = sc.nextLine();
            envioServidor.writeUTF(archivo);

            // Ahora esperamos el contenido del archivo
            String contenido = reciboServidor.readUTF();
            System.out.println("--- Contenido de " + archivo + " ---");
            System.out.println(contenido);
            System.out.println("-----");
            break;

        case "exit":
            salir = true;
            System.out.println("Desconectando...");
            break;

        default:
            // Si nos equivocamos, el servidor envía un mensaje de error
            String error = reciboServidor.readUTF();
            System.out.println("Servidor: " + error);
            break;
    }
}

catch (IOException e) {
    System.out.println("Error en la comunicación: " + e.getMessage());
}
```

Salida por terminal:

```
Servidor x Cliente x

C:\Users\tomas\.jdk\openjdk-25\bin\java.exe --enable-preview "-javaag
Servidor iniciado. Esperando credenciales de inicio de sesión...
Esperando comando del cliente...
Listando directorio...
Esperando comando del cliente...
Esperando comando del cliente...

Process finished with exit code 0
```

```
C:\Users\tomas\.jdk\openjdk-25\bin\java.exe --enable-preview "-javaag
Introduce usuario: Tomas
Introduce contraseña: 1234
¡Login correcto! Bienvenido.

Escribe un comando (ls, get, exit):
> ls

--- Archivos del Servidor ---
.gitignore
.idea
Act03_PerezCarrilloTomas.iml
ejercicio4
out
src

Escribe un comando (ls, get, exit):
> get
Introduce el nombre del archivo: ejercicio4
--- Contenido de ejercicio4 ---
Hola Guillermo.
Archivo creado para que se vea algo
de contenido
al ejecutar
el ejercicio 4

-----

Escribe un comando (ls, get, exit):
> exit
Desconectando...
```

5. Vamos a crear un servidor Multihilo UDP que va a funcionar a modo chat.

- a. Al ser UDP corremos el riesgo de que algunos mensajes se puedan perder (no tiene la fiabilidad de TCP).**
- b. El servidor va a ir recibiendo mensajes de los clientes y va a ir publicando estos en su consola de la terminal. Para identificar al cliente que le envía el mensaje vamos a publicar la dirección IP y el puerto del cliente del que vienen estos mensajes. Ejemplo: “/127.0.0.1:56122 à Hola, soy Guillermo y os animo mucho a estudiar PSP”.**
- c. El servidor debe tener alguna manera de saber si un cliente ya le ha enviado o no mensajes a ese chat, ya que la primera vez que reciba un mensaje de un cliente mostrará un mensaje del tipo: “xxxxx se ha conectado al chat” y luego mostrará el mensaje que envía el cliente. A partir de ese mensaje todos sus mensajes se publicarán normalmente.**
- d. El servidor no le contestará a los clientes salvo cuando reciba EXIT que le enviará un mensaje a dicho cliente un mensaje de gracias por participar en el Chat y que el mensaje a publicar será: “xxxxxxxxx ha abandonado el chat”**
- e. Los clientes de manera simultánea estarán enviando mensajes hasta que quieran. En el momento que no quiera enviar más mensajes enviarán EXIT y recibirán una respuesta del servidor dándole las gracias. Puesto que es posible que esta respuesta no llegue (UDP) si transcurren 10 segundos dejará de esperar dicha respuesta.**

Inicialmente, planteé guardar los nombres que llegaban en el primer mensaje dentro de un ArrayList. Sin embargo, me di cuenta de que no era una buena opción: al ser un protocolo sin conexión, si llegaban mensajes de varios clientes mezclados, el servidor no tenía forma de saber a qué nombre del array pertenecía cada paquete. Por eso reestructuré la lógica utilizando un HashMap, usando la combinación única de "IP:Puerto" como clave para identificar al usuario en cada mensaje y así evitar que se pierdan paquetes por el camino. En este ejercicio tuve que ayudarme un poco ya que esta parte del temario la tengo menos fresca. Aún así, he tratado de comentar gran parte del código para que se entienda el desarrollo más fácilmente.

Capturas de pantalla clase ServidorUDP:

```
public static void main(String[] args) { new *  
  
    int puertoServidor = 5555;  
  
    //hashmap para guardar usuarios únicos  
    HashMap<String, String> usuariosConectados = new HashMap<>();  
  
    try {  
        DatagramSocket socket = new DatagramSocket(puertoServidor);  
  
        System.out.println("Servidor UDP iniciado en el puerto " + puertoServidor);  
        System.out.println("Esperando mensajes...");  
  
        while (true) {  
            byte[] buffer = new byte[1024];  
  
            DatagramPacket paqueteRecibido = new DatagramPacket(buffer, buffer.length);  
  
            socket.receive(paqueteRecibido);  
  
            // Obtenemos la ip y el puerto para generar ID único de cada cliente  
            InetAddress direccionIP = paqueteRecibido.getAddress();  
            int puertoCliente = paqueteRecibido.getPort();  
  
            String idCliente = direccionIP.getHostAddress() + ":" + puertoCliente;  
  
            //mensaje enviado por el cliente  
            String mensaje = new String(paqueteRecibido.getData(), offset: 0, paqueteRecibido.getLength()).trim();  
  
            //El hash map está vacío así que añadimos ese usuario y mostramos el nombre recibido como primer mensaje en clase cliente  
            if (!usuariosConectados.containsKey(idCliente)) {  
                usuariosConectados.put(idCliente, mensaje); // Guardamos al usuario  
                System.out.println(mensaje + " se ha conectado al chat");  
            }  
        }  
    }  
}
```

```
    } else {  
  
        // Ya sabemos quién es, recuperamos su nombre (primer mensaje)  
        String nombreUsuario = usuariosConectados.get(idCliente);  
  
        if (mensaje.equalsIgnoreCase("EXIT")) {  
  
            System.out.println(nombreUsuario + " ha abandonado el chat");  
  
            // Enviar respuesta de despedida (Solo se contesta si es EXIT)  
            String despedida = "Gracias por participar en el Chat";  
            byte[] bufferSalida = despedida.getBytes();  
            DatagramPacket paqueteSalida = new DatagramPacket(bufferSalida, bufferSalida.length,  
                direccionIP, puertoCliente);  
            socket.send(paqueteSalida);  
  
            // Borramos al usuario de la lista de conectados  
            usuariosConectados.remove(idCliente);  
  
        } else {  
            // si es un mensaje normal pues lo mostramos  
            System.out.println("/" + idCliente + ". Nombre: " + nombreUsuario + " dice: " + mensaje);  
        }  
    }  
}
```

```
} catch (IOException e) {  
    e.printStackTrace();  
}
```

La clase ServidorUDP está dividida en dos partes: la primera recibe la clave única del cliente (ip, puerto) y recibe como valor el nombre para mostrarlo por terminal. Teniendo la clave única localizada en el hasmap es más fácil saber si el usuario ya ha enviado un mensaje previamente (nombre en clase Cliente) y así poder mostrar el resto de sus mensajes normales.

Capturas de pantalla clase ClienteUDP:

```
public static void main(String[] args) { new *

    Scanner sc = new Scanner(System.in);

    int puertoServidor = 5555;
    String hostServidor = "localhost";

    try{
        DatagramSocket socket = new DatagramSocket();
        InetAddress direccionServidor = InetAddress.getByName(hostServidor);

        //Primero le pasamos el nombre al servidor
        System.out.print("Introduce tu nombre para entrar al chat: ");
        String nombre = sc.nextLine();
        byte[] bufferNombre = nombre.getBytes();
        DatagramPacket paqueteNombre = new DatagramPacket(bufferNombre, bufferNombre.length,
            direccionServidor, puertoServidor);

        socket.send(paqueteNombre);
        System.out.println("Bienvenido. Escribe mensajes('EXIT' para salir): ");

        boolean salir = false;

        while(!salir){
            System.out.println("Escribe: ");
            String mensaje = sc.nextLine();

            byte[] bufferMensaje = mensaje.getBytes();
            DatagramPacket paqueteMensaje = new DatagramPacket(bufferMensaje, bufferMensaje.length,
                direccionServidor, puertoServidor);

            socket.send(paqueteMensaje);

            if(mensaje.equalsIgnoreCase("EXIT")) {

                //esperamos 10 segundos a recibir respuesta o cerrar conexion "forzosamente"
                socket.setSoTimeout(10000);

                try {
                    byte[] bufferRespuesta = new byte[1024];
                    DatagramPacket paqueteRespuesta = new DatagramPacket(bufferRespuesta, bufferRespuesta.length);

                    // Nos quedamos bloqueados aquí esperando respuesta
                    socket.receive(paqueteRespuesta);

                    String respuesta = new String(paqueteRespuesta.getData(), offset: 0, paqueteRespuesta.getLength());
                    System.out.println("Servidor dice: " + respuesta);

                } catch (InterruptedException e) {
                    // Esto se ejecuta si pasan 10 segundos y nadie responde
                    System.out.println("No se ha recibido respuesta del servidor (Timeout 10s). Cerrando conexión.");
                }

                salir = true;
            }
        }
    }
```

En esta clase ocurre lo mismo que en la anterior, el programa está dividido en las dos partes (pedir nombre y enviar mensajes). Por último, cuando el usuario escribe la palabra “exit” el programa se cerrará en 10s automáticamente si no recibe respuesta del servidor por el motivo que sea (InterruptedException).

Salida por terminal:

```
ServidorUDP x ClienteUDP x
C:\Users\tomas\.jdk\openjdk-25\bin\java.exe --enable-preview "-java:
Servidor UDP iniciado en el puerto 5555
Esperando mensajes...
Tomas se ha conectado al chat
/127.0.0.1:49948. Nombre: Tomas dice: Hola soy estudiante de DAM
/127.0.0.1:49948. Nombre: Tomas dice: Me gusta PSP
Tomas ha abandonado el chat
```

```
Miguelin se ha conectado al chat
/127.0.0.1:61968. Nombre: Miguelin dice: Buenas tardes a todos
/127.0.0.1:61968. Nombre: Miguelin dice: Me encanta psp
/127.0.0.1:61968. Nombre: Miguelin dice: hola mundo
Miguelin ha abandonado el chat
```

```
ServidorUDP x ClienteUDP x
C:\Users\tomas\.jdk\openjdk-25\bin\java.exe --enable-preview "-java:
Introduce tu nombre para entrar al chat: Tomas
Bienvenido. Escribe mensajes('EXIT' para salir):
Escribe:
Hola soy estudiante de DAM
Escribe:
Me gusta PSP
Escribe:
exit
Servidor dice: Gracias por participar en el Chat
Process finished with exit code 0
```

```
C:\Users\tomas\.jdk\openjdk-25\bin\java.exe --enable-preview "-java:
Introduce tu nombre para entrar al chat: Miguelin
Bienvenido. Escribe mensajes('EXIT' para salir):
Escribe:
Buenas tardes a todos
Escribe:
Me encanta psp
Escribe:
hola mundo
Escribe:
EXIT
Servidor dice: Gracias por participar en el Chat
```